

Лабораторная работа №6

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

2025-11-22

Содержание I

1. Информация

2. Введение

3. Выполнение лабораторной работы

4. Выводы

Раздел 1

1. Информация

1.1 Докладчик

► Рогожина Надежда Александровна

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы
- ▶ 1132222840@rudn.ru

1.1 Докладчик

- ▶ Рогожина Надежда Александровна
- ▶ студентка 4 курса НФИбд-02-22
- ▶ Российский университет дружбы народов им. П. Лумумбы
- ▶ 1132222840@rudn.ru
- ▶ <https://MikoGreen.github.io/ru/>

Раздел 2

2. Введение

2.1 Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

2.2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 6.2.

2.2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 6.2.
2. Выполните задания для самостоятельной работы (раздел 6.4).

Раздел 3

3. Выполнение лабораторной работы

3.1 Примеры

Первым делом импортируем нужные библиотеки:

```
In [1]: # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")
using DifferentialEquations

Updating registry at `C:\Users\Home\.julia\registries\General.toml`
Resolving package versions...
No Changes to `C:\Users\Home\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\Home\.julia\environments\v1.11\Manifest.toml`
Precompiling project...
19614.8 ms ✓ OrdinaryDiffEqNonlinearSolve
48269.9 ms ✓ BoundaryValueDiffEq
31700.0 ms ✓ OrdinaryDiffEqStabilizedIRK
34458.9 ms ✓ OrdinaryDiffEqPDIRK
38851.5 ms ✓ OrdinaryDiffEqSDIRK
47324.0 ms ✓ StochasticDiffEq
20336.1 ms ✓ OrdinaryDiffEqIMEXMultistep
40301.3 ms ✓ OrdinaryDiffEqBDF
66814.6 ms ✓ OrdinaryDiffEqFIRK
64221.4 ms ✓ OrdinaryDiffEqDefault
11587.9 ms ✓ OrdinaryDiffEq
12028.3 ms ✓ DelayDiffEq
24563.7 ms ✓ DifferentialEquations
13 dependencies successfully precompiled in 326 seconds. 494 already precompiled.
```

```
In [2]: # подключаем необходимые пакеты:
Pkg.add("Plots")
using Plots

Resolving package versions...
No Changes to `C:\Users\Home\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\Home\.julia\environments\v1.11\Manifest.toml`
```

Рисунок 1: Импорт библиотек

3.2 Примеры

In [9]: *# задаём описание модели с начальными условиями:*

```
a = 0.98
```

```
f(u,p,t) = a*u
```

```
u0 = 1.0
```

задаём интервал времени:

```
tspan = (0.0,1.0)
```

```
prob = ODEProblem(f,u0,tspan)
```

```
sol = solve(prob)
```

Out[9]: retcode: Success

Interpolation: 3rd order Hermite

t: 5-element Vector{Float64}:

0.0

0.10042494449239292

0.3521860297865888

0.6934436122197829

1.0

u: 5-element Vector{Float64}:

1.0

1.1034222047865465

1.4121908713484919

1.9730384457359198

2.6644561424814266

Рисунок 2: Модель экспоненциального роста

3.3 Примеры

```
In [5]: # строим графики:  
plot(sol, linewidth=3, title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label="u(t)", color="black")  
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")
```

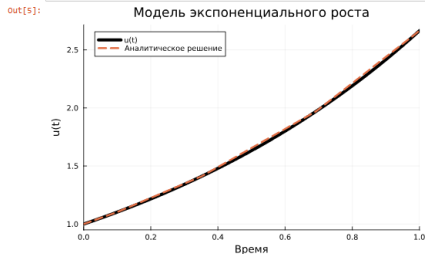


Рисунок 3: Модель экспоненциального роста. График

3.4 Примеры

```
In [11]: sol = solve(prob, abstol=1e-8, reltol=1e-8)
```

```
Out[11]: retcode: Success  
Interpolation: 3rd order Hermite  
t: 9-element Vector{Float64}:  
 0.0  
 0.04127492324135852  
 0.14679917457239627  
 0.2863153763968216  
 0.43819404263820116  
 0.6118923088547421  
 0.7985657481708535  
 0.9993514410032294  
 1.0  
u: 9-element Vector{Float64}:  
 1.0  
 1.0412786454705882  
 1.1547261208857074  
 1.3239094565289573  
 1.5363817850042807  
 1.8214892991042442  
 2.187139297776232  
 2.6627632840763393  
 2.6644562419335163
```

3.5 Примеры

```
In [12]: # строим график:  
plot(sol, lw=2, color="black", title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label="Численное решение")  
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, color="red", label="Аналитическое решение")
```

Out[12]:

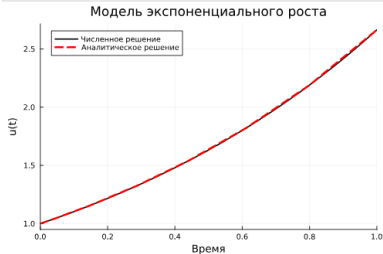


Рисунок 5: Модель экспоненциального роста. График

3.6 Примеры

```
In [13]: # задаём описание модели:  
function lorenz!(du,u,p,t)  
     $\sigma, \rho, \beta$  = p  
    du[1] =  $\sigma(u[2]-u[1])$   
    du[2] =  $u[1](\rho-u[3]) - u[2]$   
    du[3] =  $u[1]u[2] - \beta u[3]$   
end
```

```
Out[13]: lorenz! (generic function with 1 method)
```

Рисунок 6: Система Лоренца

3.7 Примеры

```
In [14]: # задаём начальные условия:
u0 = [1.0, 0.0, 0.0]

# задаём значения параметров:
p = (10, 28, 8/3)

# задаём интервал времени:
tspan = (0.0, 100.0)

# решаем:
prob = odeproblem(lorenz1, u0, tspan, p)
sol = solve(prob)

Out[14]: retcode: Success
interpolation: 3rd order Hermite
ti: 1292-element vector (Float64):
0.0
5.56786480363893488e-5
0.00039246464531993154
0.001262400751275675
0.009985876622686189
0.01936479980176763
0.027648964916420083
0.041854362219618344
0.008240411054492296
0.00365541238969924
0.11336499336803911
0.14812181393426632
0.1678207030613972
1
99.25245903911758
99.3123806179333
99.37180059024901
99.43545175573285
99.50217600308971
99.5629745172361
99.62621245212462
99.69541660424204
99.77387244642962
99.86234360861756
99.932626750918452
100.0
u: 1292-element Vector{Vector{Float64}}:
[1.0, 0.0, 0.0]
[0.999434557625185, 0.000938049817849005, 1.701434708799189e-5]
[0.9991845457425811, 0.01896339721242457, 1.160953360309339e-5]
```

Рисунок 7: Система Лоренца

3.8 Примеры

```
In [25]: pyplot()  
plot(sol, vars=(1,2,3), lw=0.5, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z", legend=False, size=(800, 500))
```

Out[25]:

Аттрактор Лоренца

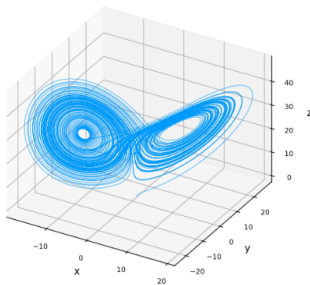


Рисунок 8: Система Лоренца. График

3.9 Примеры

```
In [27]: plot(sol, vars=(1,2,3), denseplot=false, lw=0.5, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z",  
          legend=false, size=(800, 500))
```

Out[27]:

Аттрактор Лоренца

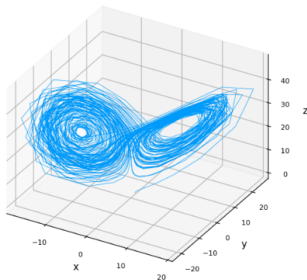


Рисунок 9: Система Лоренца. График

3.10 Примеры

```
In [28]: import Pkg
Pkg.add("ParameterizedFunctions")
using ParameterizedFunctions

113 dependencies successfully precompiled in 1211 seconds. 445 already precompiled.
114 dependencies precompiled but different versions are currently loaded. Restart Julia to access the new versions. Otherwise,
loading dependents of these packages may trigger further precompilation to work with the unexpected versions.
115 dependency had output during precompilation:
SCCNonlinearSolve
[pid 25392] waiting for IO to finish:
Handle type      uv_handle_t->data
fs_event         00000226d9913570->00000226df7cc3a0
timer            00000226e10744a0->00000226df7cc3d0
This means that a package has started a background task or event source that has not finished running. For precompilation
to complete successfully, the event source needs to be closed explicitly. See the developer documentation on fixing precompil
ation hangs for more help.
[ Info: Precompiling ParameterizedFunctions [65888b18-ceab-5e60-b2b9-181511a3b968] (cache misses: wrong dep version loaded
(2))
[ Info: Precompiling SymbolicsPreallocationToolsExt [d479e226-fb54-5ebe-a75e-a7af7f39127f] (cache misses: wrong dep version l
oaded (2))
[ Info: Precompiling BlockArraysBandedMatricesExt [231a03ec-9455-50cf-8098-89f612a0bd43] (cache misses: wrong dep version loa
ded (2))

In [29]: # задаём описание модели:
lv! = @ode_def LotkaVolterra begin
dx = a*x - b*x*y
dy = -c*y + d*x*y
end a b c d

Warning: Independent variable t should be defined with @independent_variables t.
@ ModelingToolkit C:\Users\Home\.julia\packages\ModelingToolkit\2xPD3\src\utils.jl:119

Out[29]: (::LotkaVolterra{var"###ParameterizedDiffEqFunction#275", var"###ParameterizedTGradFunction#276", var"###ParameterizedJacobia
nFunction#277", Nothing, Nothing, ODESystem}) (generic function with 1 method)
```

Рисунок 10: Модель Лотки-Вольтерры

3.11 Примеры

```
In [30]: # задаём начальное условие:
u0 = [1.0, 1.0]
# задаём значения параметров:
p = (1.5, 1.0, 3.0, 1.0)
# задаём интервал времени:
tspan = (0.0, 10.0)

prob = ODEProblem(lv!, u0, tspan, p)
sol = solve(prob)

Out[30]: retcode: Success
Interpolation: 3rd order Hermite
t: 34-element Vector{Float64}:
 0.0
 0.0776084743154256
 0.2326451370670694
 0.42911851563726466
 0.679082199936808
 0.9444046279774128
 1.2674601918628516
 1.61929140093895
 1.9869755481702074
 2.2640903679981617
 2.5125486833621876
 2.746828291948325
 3.038006876853378
 ⋮
 6.455761567904413
 6.78049521291505
 7.171039151123403
 7.5848624442719235
 7.978067891667038
 8.483164641366145
 8.719247691882519
 8.949206449510513
 9.200184762926114
 9.438028551201125
 9.711807820573478
10.0
u: 34-element Vector{Vector{Float64}}:
 [1.0, 1.0]
 [1.0454942346944578, 0.8576684823217128]
```

Рисунок 11: Модель Лотки-Вольтерры

3.12 Примеры

```
In [31]: plot(sol, label = ["Жертвы" "Хищники"], color="black", ls=[:solid :dash], title="Модель Лотки- Вольтерры",  
          xaxis="Время",yaxis="Размер популяции")
```

Out[31]:

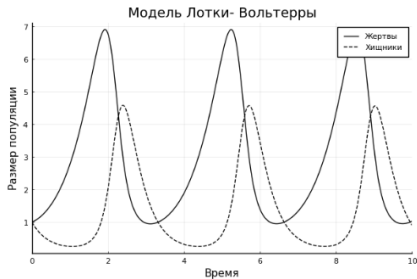


Рисунок 12: Модель Лотки-Вольтерры. График

3.13 Примеры

```
In [32]: plot(sol,vars=(1,2), color="black", хaxis="Жертвы", уaxis="Хищники", legend=false)
```

Out[32]:

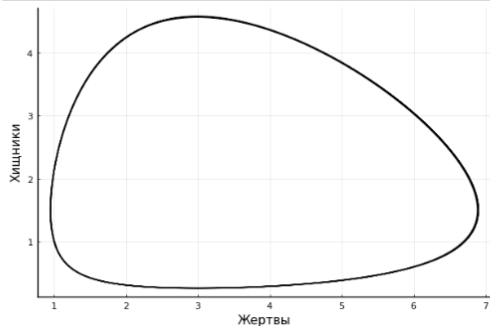


Рисунок 13: Модель Лотки-Вольтерры. Фазовый портрет

3.14 Задачи для самостоятельного выполнения

Далее, необходимо было выполнить *Задания для самостоятельного выполнения*:

3.15 Задание 1

```

In [42]: # задаем функцию модели:
function lorentz(tdu, u, p, T)
    b1, c1 = p
    du[1] = (b1 - c1)*u[1]

end
# задаем начальные условия:
u0 = [146138709]

# задаем значения параметров:
p = (2.5, 2.7)

# задаем границы времени:
tspan = (0.0, 100.0)

# решаем:
prob = ODEProblem(lorentz, u0, tspan, p)
sol = solve(prob)

Out[42]: retcode: Success
Interpolation: 3rd order Hermite
t: 33-element Vector{Float64}:
 0.0
 0.13757286814633824
 0.3613488329686421
 2.505187743977935
 2.395564239521247
 6.817948862851755
 8.299536738974452
16.87448879171369
13.874528874327985
16.60648438052495
19.06764784871468
23.19863728861737
26.64162323557912
...
68.42533481997686
64.2280375158015
68.2289482392789
72.1401594180995
76.0613879828218
78.38736643126489
83.91876222870552
87.89211281212939
91.78651188899
95.7295936841602
99.4788484816168
100.0
u: 23-element Vector{Vector{Float64}}:
 [1.46138709]

```

Рисунок 14: Решение

3.16 Задание 1

```
In [63]: plot(sol, label = ["Размер популяции"], color="blue", ls=:dash, title="Модель Мальтуса",  
           хaxis="Время", уaxis="Размер популяции")
```

Out[63]:

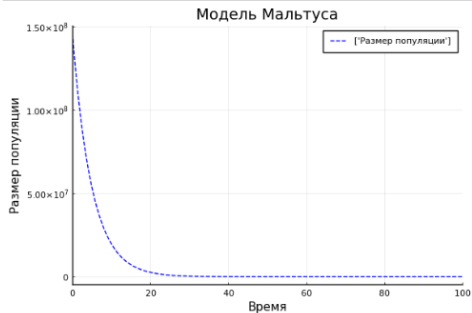


Рисунок 15: График

3.17 Задание 1

```
In [81]: plot(sol, label = ["Размер популяции"], color="blue", ls=:dash, title="Модель Мальтуса",  
           xaxis="Время", yaxis="Размер популяции", xlims=(-1, 25))
```

Out[81]:

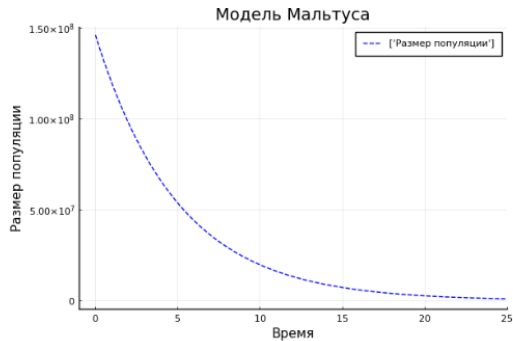


Рисунок 16: Приближение

3.18 Задание 1

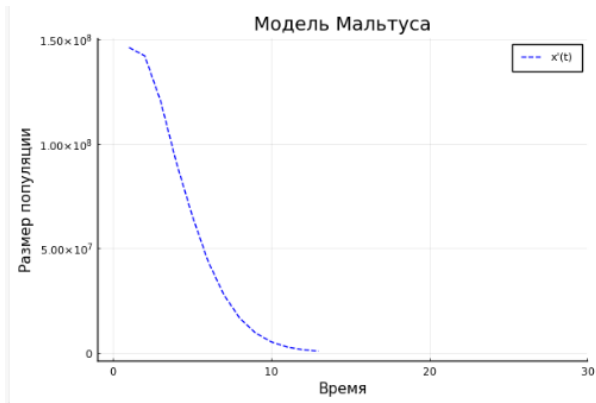


Рисунок 17: Анимация

3.19 Задание 2

```

In [95]: # задача описанная ниже:
function lorenci(du,u,p,t)
    % k = p
    du[i] = r*u[i] * (1 - u[i]/K)

end
# задача начальные условия:
u0 = [1000]

# задача начальные параметры:
p = [1,7, 3000000]

# задача интервал времени:
tspan = (0,8, 10,0)

# решатель:
prob = ODEProblem(lorenci, u0, tspan, p)
sol = solve(prob)

Out[95]: retcode: Success
interpolation: bnd order hermite
t: 20-element Vector{Float64}:
 0.0
 0.00095400211072002
 0.2427377776267187
 0.46770205682756006
 0.69049008888277709
 0.916789169648647
 1.154543362055140
 1.416431080294026
 1.70565365280470015
 2.00095114082217
 2.304646395897325
 2.622806166764106
 2.9506948397027
 3.281911641218806
 3.704051461667001
 4.127573208696704
 4.570224237085104
 5.01957067540050
 5.48111875316152
 59.0
v: 20-element Vector{Vector{Float64}}:
 [1000.0]
 [1047.315493276144]

```

Рисунок 18: Решение

3.20 Задание 2

```
In [94]: plot(sol, label = "u(t)", color="blue", ls=:dash, title="Логистическая модель роста популяции",  
            xaxis="Время", yaxis="Размер популяции")
```

Out[94]:

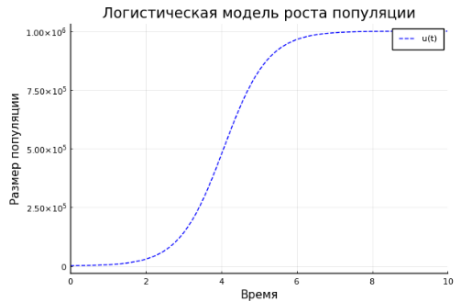


Рисунок 19: График

3.22 Задание 3

Out[167]:

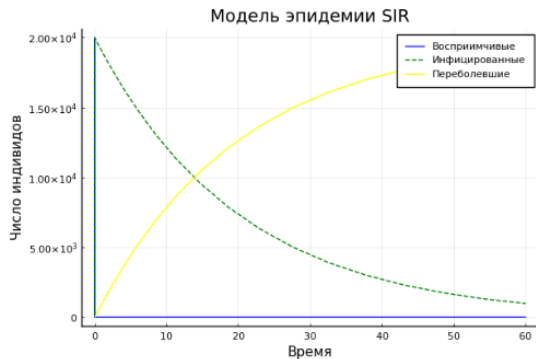


Рисунок 21: Анимация

3.23 Задание 4

```
In [169]: # задаём описание модели:
function seir(du,u,p,t)
    beta, v, d, w = p
    du[1] = beta/N * u[1] * u[3]
    du[2] = beta/N * u[1] * u[3] - d * u[2]
    du[3] = d*u[2] - v * u[3]
    du[4] = v * u[3]
end

# задаём начальные условия:
u0 = [19916, 9, 90, 5]

# задаём значения параметров:
p = (0.5, 0.05, 0.01, 20000) # beta, v, d, w

# задаём интервал времени:
tspan = (0.0, 200.0)

# решаем:
prob = ODEProblem(seir, u0, tspan, p)
sol = solve(prob)
plot(sol, color=["blue" "green" "yellow" "red"], ls=[:solid :dash :dash :solid], title="Модель эпидемии SIR",
    label=["восприимчивые" "латентно-болезнь" "инфицированные" "переболевшие"], xaxis="Время", yaxis="Число индивидов")
```

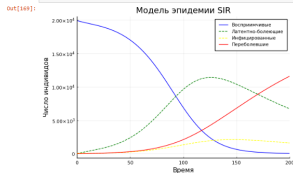


Рисунок 22: Решение + график

3.24 Задание 5

```
In [234]: # задаём описание модели:
function lotki_volterra!(du,u,p,t)
    a, c, d = p
    du[1] = a*u[1]*(1-u[1]) - u[1]*u[2]
    du[2] = -c*u[2] + d*u[1]*u[2]
end

# задаём начальные условия:
u0 = [0.3, 0.7]

# задаём значения параметров:
p = (2.0, 1.0, 5.0) # a, c, d

# задаём интервал времени:
tspan = (0.0, 100.0)

# решение:
prob = DiscreteProblem{lotki_volterra!, u0, tspan, p}
sol = solve(prob)
plot(sol, color=["blue" "green"], ls=[:solid :solid], title="Модель Лотки-Вольтерры",
     label=["x_1(t)" "x_2(t)"], xaxis="Время", yaxis="число индивидов")
```

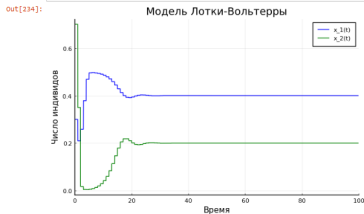


Рисунок 23: Решение + график

3.25 Задание 5

```
In [205]: plot(sol, vars=(1,2), color="red", title="Модель Лотки-Вольтерры: фазовый портрет", xaxis="Время", yaxis="Число индивидов")
```

Out[205]:

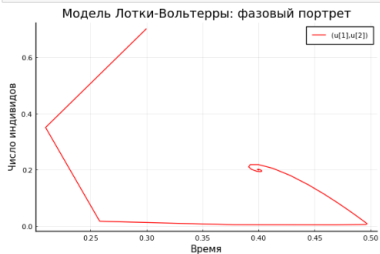


Рисунок 24: Фазовый портрет

3.26 Задание 6

```
In [216]: function competition!(du, u, p, t)
           a, b = p
           x, y = u
           du[1] = a * x - b * x * y
           du[2] = a * y - b * x * y
       end

           a = 0.2
           b = 0.5
           p = [a, b]
           u0 = [10.0, 5.0]
           tspan = (0.0, 10.0)
           prob = ODEProblem(competition!, u0, tspan, p)
           sol = solve(prob)

           # График
           plot(sol, label=["x(t)" "y(t)"], xlabel="Время t", ylabel="Численность", title="Модель конкурентных отношений")
```

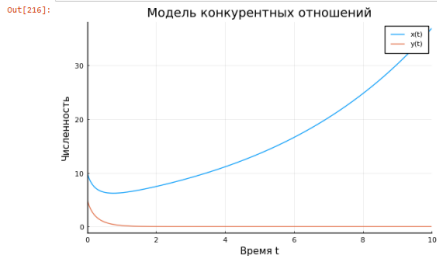


Рисунок 25: Решение + график

3.27 Задание 6

```
In [221]: anim = @animate for t in 0.0:1.0:200.0
           plot(sol.t[sol.t .<= t], reduce(hcat, sol.u[sol.t .<= t])[1, :], label="x(t)",
              xlabel="Время t", ylabel="Численность", title="Модель конкурентных отношений")
           plot!(sol.t[sol.t .<= t], reduce(hcat, sol.u[sol.t .<= t])[2, :], label="y(t)")
           end
           gif(anim, "competition.gif")

[ Info: Saved animation to C:\Users\Home\Documents\РУДН\Компьютерный практикум по статистическому анализу\lab6\competition.gif
```

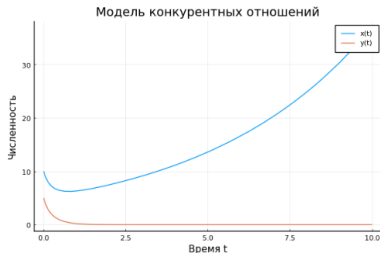


Рисунок 26: Анимация

3.28 Задание 7

Модель консервативного гармонического осциллятора

```
In [222]: function oscillator_conservative(du, u, p, t)
           u0 = p[1]
           x, v = u
           du[1] = v
           du[2] = -u0^2 * x
       end

           p = [1.0]
           u0 = [1.0, 0.0]
           tspan = (0.0, 10.0)
           prob = ODEProblem(oscillator_conservative, u0, tspan, p)
           sol = solve(prob)

           plot(sol, label=["x(t)" "v(t)"], xlabel="Время t", ylabel="Значение", title="Консервативный осциллятор")
```

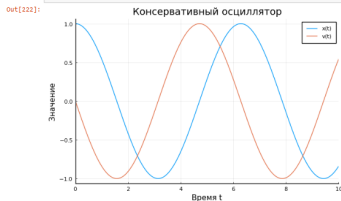


Рисунок 27: Решение + график

3.29 Задание 7

```
In [223]: # Анимация
anim = @animate for t in 0:0.1:10
    plot(sol, tspan=(0, t), label=["x(t)" "v(t)"], xlabel="Время t", ylabel="Значение",
          title="Консервативный осциллятор (анимация)", xlim=(0,10), ylim=(-2,2))
end
gif(anim, "oscillator_conservative_animation.gif", fps=10)

[ Info: Saved animation to C:\Users\Home\Documents\РУДН\Компьютерный практикум по статистическому анализу\lab6\oscillator_conse
rvative_animation.gif
```

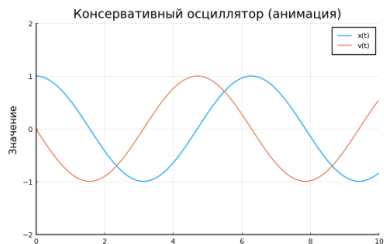


Рисунок 28: Анимация

3.30 Задание 7

```
In [224]: plot(sol[1,:], sol[2,:], label="Траектория", xlabel="x", ylabel="v", title="Фазовый портрет")
```

Out[224]:

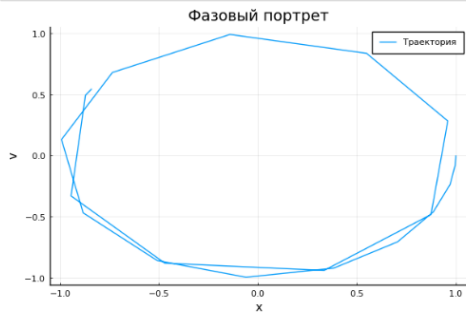


Рисунок 29: Фазовый портрет

3.31 Задание 8

Свободные колебания гармонического осциллятора

```
In [226]: function oscillator_damped!(du, u, p, t)
           w0, g = p
           x, v = u
           du[1] = v
           du[2] = -2*g*v - w0^2 * x
       end

           p = [1.0, 0.1]
           u0 = [1.0, 0.0]
           tspan = (0.0, 20.0)
           prob = ODEProblem{oscillator_damped!, u0, tspan, p}
           sol = solve(prob)
           plot(sol, label=["x(t)" "v(t)"], xlabel="Время t", ylabel="Значение", title="Затухающий осциллятор")
```

Out[226]:

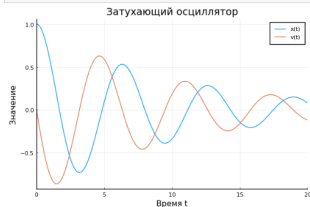


Рисунок 30: Решение + график

3.32 Задание 8

```
In [230]: anim = @animate for t in 0:0.2:20
           plot(sol, tspan=(0, t), label=["x(t)" "v(t)"], xlabel="Время t", ylabel="Значение",
               title="Затухающий осциллятор (анимация)", xlim=(0,20), ylim=(-2,2))
           end
           gif(anim, "oscillator_damped_animation.gif", fps=10)

[ Info: Saved animation to C:\Users\nome\Documents\РУДН\Компьютерный практикум по статистическому анализу\lab6\oscillator_damped_animation.gif
```

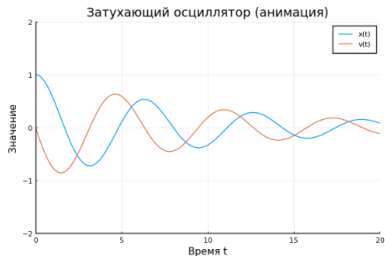


Рисунок 31: Анимация

3.33 Задание 8

```
In [229]: plot(sol, vars=(1,2), label="Траектория", xlabel="x", ylabel="v", title="Фазовый портрет")
```

out[229]:

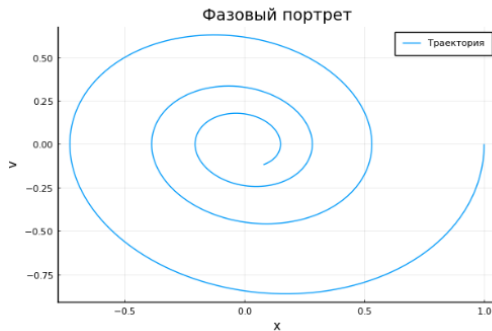


Рисунок 32: Фазовый портрет

Раздел 4

4. Выводы

4.1 Выводы

В ходе работы были изучены возможности Julia в решении дифференциальных уравнений.